

Week 6 Assignment: Trading on Real Market Data

Overview

In this assignment you will do four things:

1. **Download and inspect real daily price data**, and measure its statistical properties — including the lag-1 autocorrelation you learned to measure last week.
2. **Build a historical trading environment** that replays windows of real returns through the same interface you have used since Week 3.
3. **Train PPO on the past and evaluate it honestly on the future**, using a chronological train/test split, against the random and buy-and-hold baselines.
4. **Reflect** on the train/test gap, overfitting, and what a realistic result looks like on real data.

Setup

Install the one new dependency:

```
pip install yfinance
```

If the download fails on your network (see Common Issues), any CSV file with a date column and a closing-price column works instead — ask your mentor for one. Everything downstream only needs a 1-D array of returns.

Keep `week4_evaluate.py` in the same folder; you will import from it again.

Project Structure

By the end of Week 6, your folder should look like:

```
project_root/  
  week4_evaluate.py          (from Week 4)  
  week5_trading_env_v3.py    (from Week 5)  
  week6_get_data.py          (Task 1)  
  week6_historical_env.py    (Task 2)  
  week6_train_test.py        (Task 3)  
  week6_notes.md             (Task 4)  
  data/  
    prices.csv               (Task 1)  
    returns.npy              (Task 1)  
  plots/  
    price_and_returns.png    (Task 1)  
    test_cumulative_pnl.png  (Task 3)
```

Task 1: Get Real Data and Look at It (Core)

Last week you predicted whether real daily returns look more like $\rho = 0.5$ or $\rho = 0.0$. Time to find out. Create `week6_get_data.py`:

```

# week6_get_data.py

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf

def download_prices(ticker="SPY", start="2015-01-01",
                   end="2024-12-31", filename="data/prices.csv"):
    os.makedirs("data", exist_ok=True)
    data = yf.download(ticker, start=start, end=end, auto_adjust=True)
    close = data["Close"].squeeze().dropna()  # a pandas Series
    close.to_csv(filename)
    print(f"Saved {len(close)} rows to {filename}")
    return close

def load_prices(filename="data/prices.csv"):
    """Fallback loader: works with any CSV whose first column is a date
    and second column is a closing price."""
    df = pd.read_csv(filename, index_col=0, parse_dates=True)
    return df.iloc[:, 0].dropna()

def compute_log_returns(close):
    prices = close.to_numpy(dtype=float)
    return np.diff(np.log(prices))

def describe_returns(returns):
    lag1 = np.corrcoef(returns[:-1], returns[1:])[0, 1]
    print(f"Number of days:      {len(returns)}")
    print(f"Mean daily return:     {returns.mean():+.5f}")
    print(f"Std of daily returns:    {returns.std():.5f}")
    print(f"Lag-1 autocorrelation: {lag1:+.4f}")

def plot_price_and_returns(close, returns,
                           filename="plots/price_and_returns.png"):
    os.makedirs("plots", exist_ok=True)
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6))
    ax1.plot(close.to_numpy(dtype=float), color="steelblue")
    ax1.set_ylabel("Close price")
    ax2.plot(returns, color="darkorange", linewidth=0.5)
    ax2.set_ylabel("Daily log return")
    ax2.set_xlabel("Trading day")
    fig.tight_layout()
    fig.savefig(filename)
    plt.close(fig)
    print(f"Saved to {filename}")

if __name__ == "__main__":
    close = download_prices("SPY")
    returns = compute_log_returns(close)
    describe_returns(returns)

```

```
plot_price_and_returns(close, returns)
np.save("data/returns.npy", returns)
```

What You Must Do

1. Read through the skeleton and add comments in your own words explaining:
 - What `np.diff(np.log(prices))` computes and why log returns are used instead of raw price differences (see this week's resources).
 - What `auto_adjust=True` does and why it matters (hint: dividends and stock splits).
2. Run the script for SPY and record all four printed statistics in your notes. Compare the lag-1 autocorrelation to the ρ values from Week 5 — and to your Week 5 Question 9 prediction.
3. Compare the std of daily returns to the `daily_vol=0.01` you used in the synthetic environments. Are they in the same ballpark?
4. Optionally repeat for one more ticker you care about (e.g. RELIANCE.NS or ^NSEI for Indian markets). Does the autocorrelation change much?

Task 2: A Historical Trading Environment (Core)

Now build the environment. The observation, actions, and reward are *identical* to Week 5 — the only change is where returns come from: instead of a formula, each episode replays a random contiguous window of real history. Create `week6_historical_env.py`:

```
# week6_historical_env.py

import gymnasium as gym
from gymnasium import spaces
import numpy as np
from collections import deque

class HistoricalTradingEnv(gym.Env):
    """
    Trading environment that replays real historical returns.

    Each episode picks a random contiguous window of `episode_length`
    returns from the array passed to the constructor. Observation,
    actions, and reward are identical to TradingEnvV3 -- only the
    source of the returns has changed.

    IMPORTANT: the train/test split happens OUTSIDE this class.
    Construct one env with the training slice of returns and a
    separate env with the test slice.
    """

    def __init__(
        self,
        returns,          # 1-D numpy array of returns
        episode_length=100,
        history_len=5,
        transaction_cost=0.1,
```

```

        reward_scale=100.0,
    ):
        super().__init__()
        returns = np.asarray(returns, dtype=np.float64)
        assert len(returns) > episode_length + 1, "not enough data"
        self.returns = returns
        self.episode_length = episode_length
        self.history_len = history_len
        self.transaction_cost = transaction_cost
        self.reward_scale = reward_scale

        obs_dim = history_len + 1
        self.observation_space = spaces.Box(
            low=np.full(obs_dim, -np.inf, dtype=np.float32),
            high=np.full(obs_dim, np.inf, dtype=np.float32),
        )
        self.action_space = spaces.Discrete(3)

        self.position = None
        self.step_count = None
        self._start = None
        self._price_history = None

# ----- #
def reset(self, seed=None, options=None):
    super().reset(seed=seed)
    # Pick a random contiguous window of history for this episode
    last_valid_start = len(self.returns) - self.episode_length - 1
    self._start = int(self.np_random.integers(0, last_valid_start))
    # TODO: add a comment explaining why episodes start at random
    # positions instead of always at index 0
    self.position = 0.0
    self.step_count = 0
    self._price_history = deque(
        [0.0] * self.history_len, maxlen=self.history_len
    )
    return self._get_obs(), {}

# ----- #
def step(self, action):
    # The next return comes from HISTORY, not from a formula
    price_return = float(self.returns[self._start + self.step_count])
    self._price_history.append(price_return)

    action_to_position = {0: 1.0, 1: self.position, 2: -1.0}
    new_position = action_to_position[int(action)]

    pnl = self.position * price_return * self.reward_scale
    cost = self.transaction_cost * abs(new_position - self.position)
    reward = pnl - cost

    self.position = new_position
    self.step_count += 1
    terminated = self.step_count >= self.episode_length
    truncated = False
    return self._get_obs(), reward, terminated, truncated, {}

# ----- #
def _get_obs(self):

```

```
history = np.array(list(self._price_history), dtype=np.float32)
return np.append(history, self.position).astype(np.float32)
```

What You Must Do

1. Read through the skeleton and fill in the one `TODO` comment in `reset()` in your own words.
2. Run the usual sanity check with real data and fix any warnings:

```
import numpy as np
from gymnasium.utils.env_checker import check_env
from week6_historical_env import HistoricalTradingEnv
returns = np.load("data/returns.npy")
check_env(HistoricalTradingEnv(returns), warn=True)
```

3. Run 3 random-agent episodes and print the total reward per episode. Confirm the rewards are finite and roughly the same scale as the synthetic environments.
4. One design point to think about (for your notes): within an episode the agent sees returns from one window of history, but the *price history buffer* starts filled with zeros. What information does the agent lack during the first `history_len` steps of each episode?

Task 3: Train on the Past, Test on the Future (Core)

This is the most important discipline of the whole project: the agent trains on *earlier* data and is evaluated on *later* data it has never seen. Create `week6_train_test.py`:

```
# week6_train_test.py

import numpy as np
from stable_baselines3 import PPO
from week6_historical_env import HistoricalTradingEnv
from week4_evaluate import average_cumulative_pnl, plot_pnl

def chronological_split(returns, train_fraction=0.8):
    """First 80% of days -> training, last 20% -> testing.
    NEVER shuffle before splitting -- see this week's resources."""
    split = int(len(returns) * train_fraction)
    return returns[:split], returns[split:]

if __name__ == "__main__":
    returns = np.load("data/returns.npy")
    train_returns, test_returns = chronological_split(returns)
    print(f"Train: {len(train_returns)} days | "
          f"Test: {len(test_returns)} days")

    train_env = HistoricalTradingEnv(train_returns)
    print("Training PPO on the training years ...")
    model = PPO("MlpPolicy", train_env, verbose=0,
                learning_rate=3e-4, n_steps=512,
                batch_size=64, gamma=0.99)
    model.learn(total_timesteps=100_000)
    print("Training complete.")
```

```

# Evaluate on data the agent HAS seen (train)
# and on data it has NOT seen (test)
train_eval = HistoricalTradingEnv(train_returns)
test_eval = HistoricalTradingEnv(test_returns)

pnl_train = average_cumulative_pnl(
    train_eval, n_episodes=50, model=model, strategy="trained")
pnl_test = average_cumulative_pnl(
    test_eval, n_episodes=50, model=model, strategy="trained")
pnl_random = average_cumulative_pnl(
    test_eval, n_episodes=50, model=None, strategy="random")
pnl_bnh = average_cumulative_pnl(
    test_eval, n_episodes=50, model=None, strategy="buy_and_hold")

print("Final mean cumulative P&L over 50 episodes:")
print(f" PPO on TRAIN data: {pnl_train[-1]:+8.2f}")
print(f" PPO on TEST data: {pnl_test[-1]:+8.2f}")
print(f" Random (test): {pnl_random[-1]:+8.2f}")
print(f" Buy & hold (test): {pnl_bnh[-1]:+8.2f}")

plot_pnl(pnl_test, pnl_random, pnl_bnh,
    filename="plots/test_cumulative_pnl.png")

```

What You Must Do

1. Read `chronological_split` and add a comment in your own words: what specific mistake would shuffling the returns before splitting introduce?
2. Run the script and save `plots/test_cumulative_pnl.png`. Record all four printed numbers in your notes.
3. Compare PPO on TRAIN with PPO on TEST. A gap between them is the signature of *overfitting*: the agent memorised patterns of the training years that did not carry into the test years. How big is your gap?
4. Run the whole script two more times (each run gets fresh random seeds for training). Write down the three PPO on TEST values. How much do they move between runs? Would you have drawn a different conclusion if you had only seen one of the three?

Task 4: Written Reflection (`week6_notes.md`)

Create `week6_notes.md` and answer each question in 2–4 sentences.

On the data (Task 1):

1. What was the lag-1 autocorrelation of your real returns? How does it compare to the Week 5 markets, and what does it imply about how much edge is available to your agent?
2. How do the mean and std of real daily returns compare to the synthetic environment's `daily_vol=0.01`? Is the mean exactly zero? What does a small positive mean represent?

On the environment (Task 2):

3. Why must the train/test split be chronological rather than a random shuffle? Name the error that shuffling would introduce, and give one concrete example of how it could fake a profitable backtest.
4. Real returns are one fixed sequence — unlike Week 5, you cannot generate more. How does the random-window trick partially compensate, and what can it *not* fix?

On train vs. test (Task 3):

5. Report your four P&L numbers. Did the agent beat the random baseline on the test set? Did it beat buy-and-hold? Given Weeks 4–5, is your result surprising?
6. How large was the gap between train and test performance, and what does it tell you? If the gap is near zero, what does *that* tell you?
7. Across your three runs, how much did the test P&L vary? What does this say about trusting any single training run?
8. On many multi-year periods, buy-and-hold on an index ends clearly positive. Why is beating buy-and-hold a much higher bar than beating the random agent?

Big picture:

9. In Week 5 the agent won because $\rho \neq 0$. Real daily returns have ρ near zero — yet real quantitative funds exist. Name 2–3 places genuine structure might hide that your current state representation cannot see.
10. Write down **3 questions** about backtesting, overfitting, or RL on real data that you want to discuss with your mentor.

Optional Extension: The Week 5 Agent Meets Reality

If you are ahead of pace: load the agent you trained on the Week 5 momentum market (`model.save(...)` / `PP0.load(...)`) and evaluate it on the real test data with `average_cumulative_pnl`. It was trained for a world with $\rho = 0.5$; real data has $\rho \approx 0$. Compare its test P&L to the agent you trained on real data this week, and add a short note on what this says about training in simulators that do not match reality.

Deliverables

By the end of Week 6, you should have:

- **Code:** `week6_get_data.py`, `week6_historical_env.py`, `week6_train_test.py`.
- **Data:** `data/prices.csv` and `data/returns.npy`.
- **Notes:** `week6_notes.md` with answers to all 10 questions and the numbers from your three runs.
- **Plots:** `plots/price_and_returns.png` and `plots/test_cumulative_pnl.png`.

During your mentor meeting, walk through:

- The measured statistics of your real data — especially the autocorrelation, next to your Week 5 prediction.
- The four P&L numbers and the test plot — what is the honest conclusion?
- The spread across your three runs — how confident can you be in any of it?
- The 3 questions from Question 10 in your notes.

Looking Ahead: Weeks 7–8

The final two weeks are the capstone: you will take the pipeline you now own end to end — data, environment, training, honest evaluation — and turn it into a polished final project: improving the state features, adding risk-aware evaluation metrics (Sharpe ratio, maximum drawdown), running proper multi-seed experiments, and writing everything up. Everything you need for that already exists in your project folder after this week.

Common Issues

“yf.download fails or returns an empty DataFrame.”

Yahoo occasionally rate-limits. Wait a minute and retry, try a different ticker, or update the package (`pip install -U yfinance`). If your network blocks it entirely, ask your mentor for a CSV and use `load_prices()` instead of `download_prices()` — the rest of the week is unchanged.

“data[“Close”] is a DataFrame, not a Series.”

Recent yfinance versions return multi-level columns when downloading. The `.squeeze()` in the skeleton handles the single-ticker case; if you downloaded multiple tickers, select one with `data[“Close”][ticker]`.

“My returns contain NaN.”

Drop missing prices *before* computing returns (`close.dropna()`, already in the skeleton). If NaNs appear after `np.diff(np.log(...))`, check for zero or negative prices in your CSV.

“Train P&L is clearly positive but test P&L is near zero.”

That is not a bug — that is the lesson. The agent memorised patterns of the training years that did not generalise. Say so plainly in your notes; an honest null result, properly evaluated, is a perfectly good outcome for this week.

“Buy-and-hold beats my agent on the test set.”

Common on periods where the market trended up: buy-and-hold collects the market drift while paying zero transaction costs. Being long the drift is a strategy your agent must *discover*, and with costs it may not get there. Note it and discuss it with your mentor rather than tuning until the “problem” disappears.